DECODELABS

AI Project 2

Data Classification Using AI

Complete Step-by-Step Roadmap + Prerequisites

Batch 2026 | Intern Handbook

Project	Project 2 — Data Classification Using AI
Algorithm	K-Nearest Neighbors (KNN)
Dataset	Iris Benchmark (150 samples, 3 classes, 4 features)
Framework	Python + scikit-learn
Deliverable	Trained model + Confusion Matrix + F1 Score report
Mandatory	Must complete to unlock next week's projects

SECTION 1 — PREREQUISITES

A. Knowledge You Should Have

Python basics	Variables, loops, functions, importing libraries
Data intuition	Understanding rows (samples) and columns (features) in a table
Basic math	Averages, distances — no advanced calculus needed
NOT required	Deep learning, neural networks, advanced statistics

B. Software to Install

Install **Python 3.8+** from **python.org** if you haven't already.

Then install all required libraries in one command:

```
pip install numpy pandas scikit-learn matplotlib seaborn
```

numpy	Numerical operations & array handling
pandas	Data loading, exploration, and manipulation
scikit-learn	ML algorithms, train-test split, scaling, metrics
matplotlib	Plotting graphs and confusion matrix heatmaps
seaborn	Enhanced statistical visualisations

C. Recommended IDE / Environment

Google Colab	BEST for beginners — zero install, runs in browser, all libs pre-loaded. Go to colab.research.google.com
Jupyter Notebook	Install with: pip install notebook then run: jupyter notebook
VS Code	Full IDE. Download from code.visualstudio.com. Install the Python extension.

■ *Recommendation for antigravity: Use Google Colab — no setup, just open and code!*

D. Dataset — No Download Needed!

The Iris dataset is built into scikit-learn. Load it with one line:

```
from sklearn.datasets import load_iris
iris = load_iris()
```

SECTION 2 — STEP-BY-STEP PROJECT ROADMAP

1

Load & Explore the Dataset

Import the Iris dataset and convert it to a pandas DataFrame so you can view and understand the data before doing anything else.

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

print(df.head()) # first 5 rows
print(df.describe()) # statistics
print(df['species'].value_counts()) # class balance
```

What to look for: 50 samples per class (balanced), 4 numeric features, no missing values.

2

Separate Features (X) and Labels (y)

X contains the input measurements; y contains the correct class labels. The model learns the mapping from X to y.

```
X = iris.data # shape (150, 4)
y = iris.target # 0=Setosa, 1=Versicolor, 2=Virginica
```

Features (4): Sepal Length, Sepal Width, Petal Length, Petal Width — all in cm.

3

Apply Feature Scaling (StandardScaler)

KNN uses **distance** to find neighbours. If one feature has values 0–1000 and another 0–1, the larger-scale feature dominates unfairly. Scaling fixes this by making every feature have Mean=0 and Variance=1.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X) # fit on X then transform
```

■ *Always fit the scaler on training data only (after the split). Fitting on the full data leaks test information.*

4 Split Data — Train (80%) / Test (20%)

The training set teaches the model. The test set — which the model has never seen — validates it. Shuffle first to remove order bias.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
X_scaled, y,
test_size=0.2, # 20% for testing
random_state=42, # reproducibility
shuffle=True # remove order bias
)

print(X_train.shape) # (120, 4)
print(X_test.shape) # (30, 4)
```

5 Build & Train the KNN Model

KNN stores the training data and, at prediction time, finds the K nearest neighbours by Euclidean distance and takes a majority vote.

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5) # start with K=5
model.fit(X_train, y_train) # memorise the map

predictions = model.predict(X_test) # apply logic
```

Why K=5? K=1 overfits (memorises noise). K=100 underfits (too generic). K=5 is a safe starting point. You will tune it in Step 7.

6

Evaluate — Confusion Matrix & F1 Score

Never rely only on accuracy — in imbalanced datasets it is misleading (the "99% Accuracy Mirage"). Use the Confusion Matrix and F1 Score.

```
from sklearn.metrics import (classification_report,
confusion_matrix, ConfusionMatrixDisplay)
import matplotlib.pyplot as plt

# Text report
print(classification_report(y_test, predictions,
target_names=iris.target_names))

# Confusion matrix heatmap
cm = confusion_matrix(y_test, predictions)
disp = ConfusionMatrixDisplay(cm, display_labels=iris.target_names)
disp.plot(cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```

TP — True Positive	Model said YES and it was YES (correct!)
TN — True Negative	Model said NO and it was NO (correct!)
FP — False Positive	Model said YES but it was NO (false alarm / Type I)
FN — False Negative	Model said NO but it was YES (missed / Type II)
Precision	Of all predicted positives, how many were real? (trustworthiness)
Recall	Of all real positives, how many did we catch? (sensitivity)
F1 Score	Harmonic mean of Precision and Recall — the balanced metric

7 Tune K — Find the Optimal Neighbour Count

Test multiple K values and plot error rate vs K. The optimal K sits at the **"elbow"** — the point where error stops decreasing significantly.

```
from sklearn.metrics import f1_score
import matplotlib.pyplot as plt

k_values = range(1, 21)
f1_scores = []

for k in k_values:
m = KNeighborsClassifier(n_neighbors=k)
m.fit(X_train, y_train)
preds = m.predict(X_test)
f1_scores.append(f1_score(y_test, preds, average='weighted'))

plt.plot(k_values, f1_scores, marker='o', color='steelblue')
plt.xlabel('K value')
plt.ylabel('F1 Score')
plt.title('Tuning K — Find the Elbow')
plt.grid(True)
plt.show()

best_k = k_values[f1_scores.index(max(f1_scores))]
print(f'Best K: {best_k}')
```

■ *Retrain your final model using best_k to get the best possible predictions.*

8 Final Model — Retrain with Best K & Report

Use the best K found in Step 7. Print the final classification report to confirm your F1 Score and submit.

```
final_model = KNeighborsClassifier(n_neighbors=best_k)
final_model.fit(X_train, y_train)
final_preds = final_model.predict(X_test)

print('=== FINAL RESULTS ===')
print(classification_report(y_test, final_preds,
target_names=iris.target_names))
```

Target: Aim for an F1 Score above 0.90 on the Iris dataset. Scores of 0.95–1.00 are common with optimal K and proper scaling.

SECTION 3 — QUICK REFERENCE CHEAT SHEET

load_iris()	Load the built-in Iris dataset
pd.DataFrame()	Convert dataset to a readable table
train_test_split()	Split data into train / test sets
StandardScaler()	Normalise features (Mean=0, Var=1)
KNeighborsClassifier()	Create the KNN classifier
model.fit(X_train, y_train)	Train (memorise) the model
model.predict(X_test)	Generate predictions on unseen data
confusion_matrix()	Table of TP / FP / FN / TN
classification_report()	Precision, Recall, F1 per class
f1_score(..., average='weighted'	Weighted F1 across all classes

Common Mistakes to Avoid

Fitting scaler on full dataset	Fit ONLY on X_train, then transform both X_train and X_test separately.
Using accuracy alone	Always check F1 Score + Confusion Matrix for full picture.
Forgetting to shuffle	set shuffle=True in train_test_split to avoid order bias.
Choosing K=1	K=1 overfits — always tune K using the elbow method.
Not setting random_state	Set random_state=42 so results are reproducible.

What Comes After Project 2?

Project 3+	Deep Learning & Convolutional Neural Networks (CNNs)
Data type	Move from tabular data to image (pixel) data
Key concept	Computer Vision — teaching machines to see

■ *Remember: every low accuracy score is a learning opportunity. Experiment with different K values, compare algorithms, and test with new data!*